

VAYU-NET

Test Plan & Validation Procedures

Backend, data and UI/UX validation for the multi-agent urban air quality platform

Delhi · Mumbai · Bengaluru

This plan defines what must be true before VAYU-NET is handed to an operating agency, and the exact procedure for proving each of those things. It covers five test levels, 126 numbered cases, and a regression suite built from defects that reached production once.

field	value
Generated	2026-08-29
Companion document	VAYU-NET_System_Design.pdf — architecture, schema, API reference
Automated backend suite	deploy/verify_live.py — 17 checks
Automated estimator suite	deploy/verify_ward_estimate.mjs — 12 assertions
Frontend	https://vayu-net-ten.vercel.app
API	https://vayu-net-api-ver-tex.vercel.app

The governing principle of this plan. VAYU-NET makes two kinds of claim that ordinary software does not: it tells a citizen whether the air they are about to breathe is dangerous, and it tells an officer which statute to cite. A defect in either is not a bug report — it is wrong health advice or an unsound legal claim. Test priority follows that, not code coverage.

Contents

§	section	cases
1	Scope, objectives and what 'correct' means here	—
2	Test environments and the production-testing gap	—
3	Risk model and defect severity	—
4	Backend — data layer invariants	DATA-01 ... 16
5	Backend — agent chain correctness	CHAIN-01 ... 14
6	Backend — API contract	API-01 ... 22
7	UI/UX — Commissioner journey	UIC-01 ... 14
8	UI/UX — Resident journey	UIR-01 ... 16
9	UI/UX — cross-cutting: accessibility, responsive, states, i18n	UX-01 ... 18
10	Regression suite — defects that shipped once	REG-01 ... 14
11	Non-functional — performance, resilience, security	NFR-01 ... 09, SEC-01 ... 03
12	Entry and exit criteria, sign-off	—
13	Gaps in testability — what this plan cannot prove	—

1 · Scope, objectives and what 'correct' means here

In scope

The deployed platform end to end: hourly ingestion, the five-stage agent chain, the Postgres store, all fourteen API endpoints, both user interfaces (war-room and citizen), the evidence-pack PDF generator, and the multilingual advisory path.

Out of scope

Third-party availability (data.gov.in, Open-Meteo, NASA FIRMS, Anthropic, Vercel, Supabase) is not under test — but the platform's *behaviour when they fail* is, and is covered in §11. Load testing beyond a single concurrent officer is deferred: there is no multi-tenancy yet, so there is no realistic concurrency model to test against.

Four definitions of correct, in priority order

#	the platform must never...	why this ranks here
1	...state a measured value it did not measure, or attach a health instruction to a number whose provenance it cannot support.	This is the only output a member of the public acts on directly, without an expert in the loop.
2	...print a statutory citation over air that does not warrant enforcement, or rank a ward it cannot justify.	An unsound legal claim damages the agency that issued it, and the platform's credibility is its entire value.
3	...present stale data as current, or a build artefact as a live reading.	Both have happened. A freshness claim is load-bearing for every other claim on the screen.
4	...lose an hour of ingestion silently.	data.gov.in serves only the current hour and there is no backfill, so a gap is permanent and degrades detection for days.
Everything else — layout, latency, polish — is subordinate. A test failure in category 1 or 2 blocks release regardless of how minor it looks.		

2 · Test environments and the production-testing gap

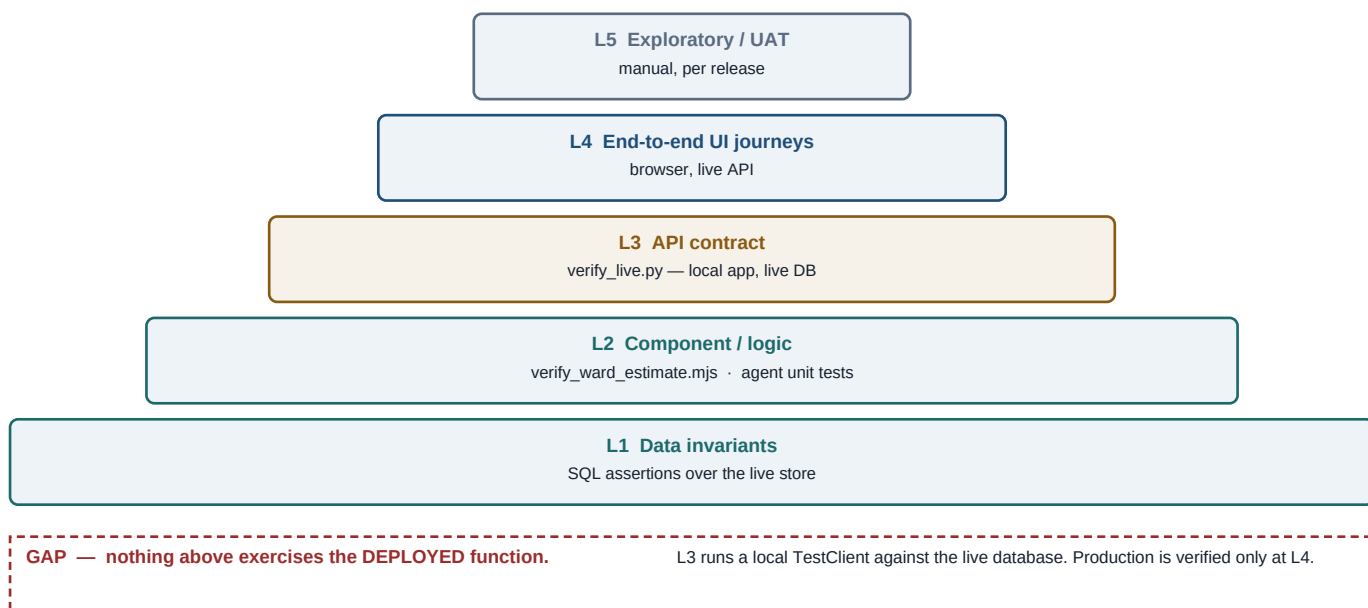


Figure 1 — Test levels. The dashed band is the coverage gap this plan exists to close.

The single most important thing a tester must know about this platform: `deploy/verify_live.py` builds a `TestClient(app)` from the `local service/live_api.py` and points it at the *live* database. It therefore validates local code against production data and never calls the deployed Vercel function. It has already reported ALL PASSED while the deployed API was serving materially different behaviour.

Consequence for this plan: **every API case in §6 must be run twice** — once via the suite (fast, catches logic regressions) and once with `curl` against the deployed URL before sign-off. Only the second run is evidence about production.

Environments

environment	frontend	API	database	used for
Local dev	vite dev server, VITE_API_URL set	deployed API	live Supabase	UI development, L4 rehearsal
Local backend	n/a	TestClient(app)	live Supabase	L1–L3 automated suites
Production	vayu-net-ten.vercel.app	vayu-net-api-ver-tex.vercel.app	live Supabase	L4 sign-off, smoke tests

There is one database. No staging instance exists, so every test above runs against production data. Two rules follow, and they are not optional:

- 1 · Never run write-path tests casually.** `POST /ingest` and `POST /replay/run` both rewrite `attributions` and `truncate/rebuild` actions. They are safe to run (the chain is designed to be re-runnable) but they change what every other tester sees mid-session. Coordinate, and record the `run_id`.
- 2 · Take a backup before any destructive test.** A full parquet dump of all nine tables already exists at `_backup/*.parquet`; refresh it first.

3 · Risk model and defect severity

Severity is assigned by what the defect causes a person to believe, not by how much code is involved. A one-character CSS change that makes an estimate look measured is Sev-1.

sev	definition	example from this platform's history	release
1	The platform asserts something false about air quality or law, to a user who will act on it.	Every sensor-less ward displayed the city mean of 130 as if measured, while the sensor 1.3 km away read 448 Severe.	Blocks
2	A core function is unavailable or silently wrong, but the error is visible or bounded.	The Events tab rendered the oldest 80 events instead of the newest, so recent activity was invisible.	Blocks
3	Degraded behaviour with a correct fallback, or a defect confined to a non-critical view.	Metrics tab showed blank 48h/72h rows because the inventory table was excluded from the bundle.	Fix before pilot
4	Cosmetic, or an inconvenience with an obvious workaround.	Map goes blank after repeated reloads in one tab — a WebGL context limit, not a platform fault.	Backlog

Risk-ranked areas

area	risk	why	depth
Ward AQI provenance	Critical	Directly drives public health advice; 87% of ward values are interpolated	Exhaustive — \$8, \$10
Enforcement floor and ranking	Critical	Produces statutory citations	Exhaustive — \$5
Freshness reporting	High	Every other claim depends on it; has failed before	Full — \$4, \$6
Ingest continuity	High	Gaps are permanent and degrade detection for days	Full — \$5, \$11
Attribution confidence	High	Shown as a percentage; users will read it as precision	Full — \$5
Translation fidelity	Medium	Validated with fallback, but a mistranslated band is a health error	Sampled — \$9
Map rendering	Low	Read-only visualisation with a table equivalent	Smoke — \$7

4 · Backend — data layer invariants

Run against the live store. These hold at all times, not merely after a pipeline run. **No FOREIGN KEY constraints are declared anywhere in this schema**, so referential integrity is not enforced by the database and must be asserted here — that is precisely how the duplicate-station-identity defect stayed invisible.

id	validates	procedure	expected result	auto
DATA-01	readings_hourly idempotency	Run POST /ingest twice for the same hour. Compare count(*) before and after the second run.	Row count unchanged. The (station_id, h) primary key makes the repeat an update.	yes
DATA-02	No band on a null AQI	GET /cities/delhi/stations; filter rows where aqi is null.	Every such row has band = null. A NaN must never fall through to 'Severe'.	yes
DATA-03	No duplicate sensor identity	Group stations by lower(trim(station_name)); look for names mapping to >1 station_id.	Zero duplicates. Two id schemes for one sensor halve the ward mapping.	yes
DATA-04	Ward mapping coverage	GET /cities; compare stations vs mapped per city.	Delhi 46/46. Any unmapped sensor is reported, never silently dropped.	yes
DATA-05	No orphan ward references	SQL: stations left join wards on ward_id where ward_id is not null and wards.ward_id is null.	Zero rows. No FK exists, so this must be asserted.	no — add
DATA-06	No orphan attributions	SQL: attributions left join stations using (station_id) where stations.station_id is null.	Zero rows.	no — add
DATA-07	Confidence bounds	SQL: select min(confidence), max(confidence) from attributions.	Within [0, 100]. A softmax output outside this indicates a scoring defect.	no — add
DATA-08	Category domain	SQL: select distinct category from attributions.	Subset of the five allowed categories. No nulls, no unexpected values.	no — add
DATA-09	No zero-priority action	GET /cities/{c}/actions for all three cities.	Every priority > 0. Sub-floor wards are dropped, never ranked at 0.00.	yes
DATA-10	Era domain and split	SQL: select distinct era from actions; check both pools ranked independently.	Only 'live' and 'episode'. Each capped at 10 per city.	yes
DATA-11	actions is a projection, not a log	Record max(action_id); run the chain; re-read.	action_id restarts from 1. The table is DELETED and rebuilt — it must not grow unbounded.	no — add
DATA-12	fires idempotency	Re-run etl/fetch_fires_live.py; compare row count.	No duplicate detections; (h, latitude, longitude) is the key.	no
DATA-13	llm_spend is a singleton	SQL: select count(*) from llm_spend.	Exactly 1 row, id = 1. More than one means the cap can be bypassed.	no — add
DATA-14	ops_config is locked	Check RLS enabled and anon/authenticated grants revoked.	RLS on, no policy, no grants. This table holds the ingest token.	yes
DATA-15	Timezone sanity	SQL: select max(h) from readings_hourly; compare with (now()) at time zone 'Asia/Kolkata'.	max(h) is in the past. A future timestamp means the 5h30m skew has returned.	no — add
DATA-16	RLS parity across tables	Query pg_class.rerowsecurity for all nine tables.	All nine enabled. Currently warns: fires and llm_spend are RLS-off — reported, not failed, while the item is deferred; see §11 SEC-03.	yes — warn

Seven of these sixteen are not yet automated. They are the highest-value additions to `verify_live.py`, because each asserts an invariant the database itself does not enforce.

5 · Backend — agent chain correctness

These are the tests that decide whether an enforcement recommendation is defensible. Several require synthetic input: build a small fixture DataFrame rather than waiting for the weather to cooperate.

id	validates	procedure	expected result	auto
CHAIN-01	Relative-spike detection	Fixture: flat 40 µg/m³ series, then one hour at 200. Call detect_events().	Exactly one relative_spike event, z-score > 2.5.	no
CHAIN-02	Median/MAD robustness	Fixture with a single 600 µg/m³ outlier inside the 48h window, then a genuine spike.	The outlier does not inflate the baseline enough to mask the later spike. Mean/σ would fail this.	no
CHAIN-03	Severe crossing is edge-triggered	Fixture: PM2.5 rises 240 → 260 → 280 → 300.	One severe_crossing at the 240→260 transition only. Not one per hour above 250.	no
CHAIN-04	Baseline lag	Fixture where the current hour is the largest value seen.	The current reading is excluded from its own baseline (shift(1)), so it can still trip.	no
CHAIN-05	Log-odds cap enforced	Construct an event where one signal is overwhelmingly strong.	That signal's contribution clips at 1.5; it cannot alone drive confidence to ~100%.	no
CHAIN-06	Confidence is a distribution	Attribute any event; sum the per-category scores.	Sums to 100 across the five categories. Reported confidence is the argmax share.	no
CHAIN-07	No model call in the hot path	Record llm_spend.calls; run the full chain; re-read.	calls unchanged. The automated cycle is template-only and must spend nothing.	no — add
CHAIN-08	Enforcement floor — below	Fixture ward with mean_pm25 = 59.	No action produced. Not an action at priority 0.	no
CHAIN-09	Enforcement floor — above	Same fixture at mean_pm25 = 61.	Exactly one action, priority > 0, with a statute attached.	no
CHAIN-10	Era isolation	Inspect live-pool actions after a run.	No action in the live pool has last_seen older than the 3-day live cutoff. December cannot outrank the present.	yes
CHAIN-11	Vulnerability bounds	SQL: select min(vulnerability), max(vulnerability) from actions.	Within [1.0, 1.5]. Outside that, the normalisation has broken.	no
CHAIN-12	Persistence monotonicity	Two fixture wards, identical but for event duration (2h vs 20h).	The longer-lived ward ranks higher, all else equal.	no
CHAIN-13	Re-run does not duplicate	Run the chain twice; count attributions in the live window.	Count is stable. The live window is deleted then reinserted.	no — add
CHAIN-14	Empty input is survivable	Run the chain against a window with zero detected events.	Returns cleanly with events: 0. No exception, no partial write, actions left empty rather than stale.	no

Audit-trail check (manual, every release). Pick any action shown in the UI. Using its run_id from /agent_log, walk back: action → the attributions it aggregated → the readings those events came from. Every number on screen must reconcile to a stored row. If any step cannot be traced, that is Sev-1 — the audit trail is the platform's core claim.

6 · Backend — API contract

Run each case twice: against `TestClient` via the suite, and with `curl` against the deployed URL. Divergence between the two is itself a Sev-2 finding — it means production is running different code.

id	validates	procedure	expected result	auto
API-01	Health reports true store state	GET /health.	ok:true, store:'supabase', a readings count, newest_reading, age_hours from the DB clock.	yes
API-02	Freshness is not a build artefact	Compare /health.newest_reading with SQL max(h).	Identical. The banner once showed build time, 12 days stale.	yes
API-03	Ingest configuration is observable	GET /health; read ingest_configured and ingest_token_source.	Source is one of env ops_config unset unavailable — distinguishes 'saved' from 'visible to the code'.	yes
API-04	Station payload shape	GET /cities/delhi/stations.	Each row carries station_id, station_name, lat, lon, ward_id, pm25, pm10, as_of, aqi, band.	yes
API-05	Events are newest-first	GET /cities/delhi/events; inspect the first and last h.	Descending by h. The UI slices the head, so ordering is load-bearing.	yes
API-06	Event limit honoured	GET /cities/delhi/events?limit=10.	Exactly 10 rows.	no
API-07	Actions ordered by priority	GET /cities/delhi/actions.	Descending priority, all > 0.	yes
API-08	Era filter	GET /cities/delhi/actions?era=live and ?era=episode.	Disjoint sets; each ≤10 per city.	yes
API-09	Compare computed in SQL	GET /compare; check Delhi's event count.	Not exactly 300. Client-side derivation once hit the page cap and always reported 300.	yes
API-10	Metrics completeness	GET /metrics.	live_store block populated; no blank 48h/72h rows; fires status present.	yes
API-11	Evidence pack renders	GET /actions/{id}/pack.pdf for a valid id.	A non-trivial PDF (>10 KB), content-type application/pdf.	yes
API-12	Evidence pack — bad id	GET /actions/999999/pack.pdf.	A clean error, not a stack trace or a zero-byte PDF.	no
API-13	Ingest rejects no token	POST /ingest with no header.	401. Never 200.	yes
API-14	Ingest rejects wrong token	POST /ingest with a wrong bearer token.	401, constant-time comparison.	yes
API-15	Ingest disabled without config	Unset both token sources in a scratch environment; POST /ingest.	503 with an explanatory error — it must not run unauthenticated.	no
API-16	Token never in the query string	Review access patterns and docs.	Header-only. A query-string token is logged by every proxy in the path.	manual
API-17	Advisory — measured basis	GET /advisory?...&basis=current.	Text contains 'measured now'; response echoes basis:'current'.	yes
API-18	Advisory — estimated basis	GET /advisory?...&basis=estimated.	Text says 'an estimated AQI ...', interpolated'; never the word 'measured'; echoes basis.	yes
API-19	Advisory — unknown basis degrades safely	GET /advisory?...&basis=nonsense.	Degrades to 'estimated', never to 'current'. Overstating certainty is the harmful direction.	yes
API-20	Advisory — AQI survives every basis	Render the template for all three bases.	The AQI digits appear in all three. The validator drops any translation that loses them.	yes
API-21	Translation validation	Request lang=hi with a correct Devanagari sample; then one with a wrong number.	Correct one passes even with ■■■ numerals; wrong number rejected.	yes
API-22	CORS posture	Inspect Access-Control-Allow-Origin.	Currently '*'. Documented as an open gap — see SEC-01.	manual

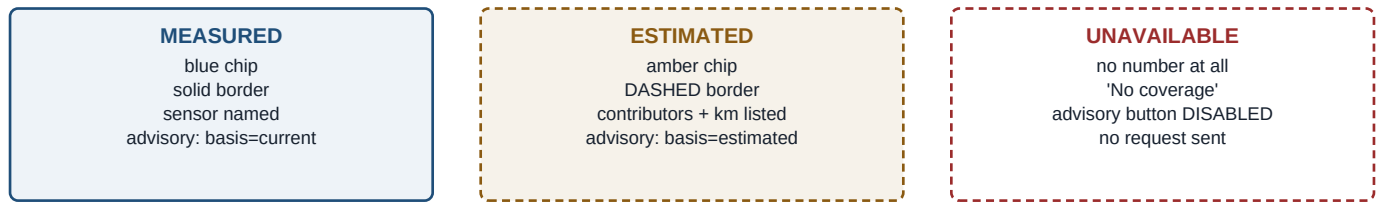
7 · UI/UX — Commissioner journey

Persona: a pollution control board officer deciding where to send an inspection team today. Test on the deployed site, in a fresh tab. Repeated reloads in one tab exhaust WebGL contexts and the map goes blank — a browser artefact, not a defect.

id	validates	procedure	expected result	auto
UIC-01	Freshness banner truthfulness	Load the site; compare the banner against /health.age_hours.	Age matches the API, stated as a real age ('under an hour old'), never as a promised cadence.	no
UIC-02	City switch refetches	Switch Delhi → Mumbai → Bengaluru.	Map recentres; all tabs repopulate; no stale rows from the previous city.	no
UIC-03	Era toggle changes the pool	Toggle Live ↔ Crisis episode on Actions.	Content changes; live shows recent wards, episode shows the December set.	no
UIC-04	Action card completeness	Open Actions for Delhi, live era.	Each card shows priority, ward, category badge, confidence %, event count, mean/max PM2.5, the intervention and the statute.	no
UIC-05	Priority ordering is visible	Read the priority values top to bottom.	Monotonically decreasing. An officer must be able to trust the first card is the top priority.	no
UIC-06	Evidence pack opens and reconciles	Click Evidence pack on the top action.	PDF opens; ward, category, PM2.5 figures and statute match the card exactly.	no
UIC-07	Empty state reads as an answer	Open Actions for Bengaluru, live era.	An explicit 'no enforcement action in the last 72 hours' message with the reason — not a blank panel that looks broken.	no
UIC-08	Events newest-first	Open Events; read the timestamps.	Most recent at the top. This regressed once to oldest-first.	no
UIC-09	Live hotspots appear above events	Events tab, live era, Delhi.	Stations ≥ AQI 201 listed above the event cards, labelled as needing no history.	no
UIC-10	Evidence bullets render	Expand any event card.	Up to three evidence bullets from evidence_json; no raw JSON leaking into the UI.	no
UIC-11	Compare is genuinely comparable	Open Compare.	All three cities scored on one formula; the note explaining identical terms is present.	no
UIC-12	Replay reports latency	Run Replay.	Returns a timed log ending with signal_to_action_seconds; UI shows it ran live, not precomputed.	no
UIC-13	Stale sensors are visually distinct	Inspect map dots in live era.	Archive-only stations render smaller and greyed, never as a live coloured reading.	no
UIC-14	Attribution disclaimer present	Check the footer and any category badge.	'Evidence-weighted likelihood with confidence, not legal proof' visible. This must never be removed.	no

8 · UI/UX — Resident journey

Persona: a resident checking whether it is safe to send a child outside. This is the highest-risk surface in the platform: the advice is acted on immediately, by someone with no way to audit it.



FAIL CONDITION: any two of these three are visually indistinguishable at a glance, or a number appears under UNAVAILABLE.

Figure 2 — Acceptance criteria for the ward reading panel. Visual distinctness is a requirement, not styling.

id	validates	procedure	expected result	auto
UIR-01	Measured ward	Select 'I.P EXTENTION — Anand Vihar'.	MEASURED chip, solid border, value matches the map dot and /stations exactly, sensor named.	no
UIR-02	Estimated ward	Select 'ANAND VIHAR' (the ward, which holds no sensor).	ESTIMATED chip, dashed amber border, 'No sensor in this ward', contributor list with distances, confidence stated.	no
UIR-03	Estimate is not the city mean	Compare several sensor-less wards.	Values differ across wards. One number repeated everywhere is the regression this replaced.	yes
UIR-04	Worse neighbour is surfaced	Select a ward adjacent to a Severe sensor.	Red panel naming that sensor, its AQI, band and distance, advising the worse guidance.	no
UIR-05	No-coverage refusal	Select a ward with no sensor within 8 km.	No number at all. 'No coverage'. Advisory button disabled.	yes
UIR-06	Disabled button explains itself	On a no-coverage ward, look for the reason.	The panel states why no advisory can be issued; the disabled control is not unexplained.	no
UIR-07	Landmark in the ward list	Open the ward dropdown.	Wards holding a sensor show 'WARD NAME — Sensor'. Municipal charge names alone are unsearchable.	no
UIR-08	Advisory matches provenance	Get an advisory on an estimated ward.	Text says 'an estimated AQI ...', interpolated'. The word 'measured' must not appear.	no
UIR-09	Version-skew guard	Point the client at an API predating the basis field.	Client falls back to the local CPCB template; source reads 'API did not confirm basis'.	no
UIR-10	Audience group changes guidance	Cycle schools / outdoor workers / elderly / general at the same AQI.	Guidance text changes appropriately; band-specific wording is CPCB's, not generated.	no
UIR-11	Hindi advisory	Request lang=hi.	Devanagari renders; the AQI number is present and correct in any numeral system; source is llm_translated.	no
UIR-12	Marathi and Kannada	Repeat for mr and kn.	Correct script, correct number, no fallback to English unless the model failed.	no
UIR-13	Translation failure falls back safely	Simulate a model error or exhausted budget.	English CPCB template shown, clearly sourced. Never a blank advisory or a partial translation.	no
UIR-14	Text-to-speech	Press Listen for each language.	Speech uses the matching locale (hi-IN, mr-IN, kn-IN). Wrong locale renders the text unintelligible.	no
UIR-15	Stale sensors excluded	Check a ward whose only sensor is archive-only.	That sensor does not contribute. Winter values once appeared behind a monsoon-season advisory.	no
UIR-16	Ward POI context	Read the line under the ward selector.	School and hospital counts shown with OpenStreetMap attribution.	no

9 · UI/UX — cross-cutting validation

Accessibility

This platform will be used by a public body and must be usable by the public. The current design encodes AQI severity **primarily in colour**, which is the most common accessibility failure in air-quality interfaces and is a genuine finding here, not a hypothetical.

id	validates	procedure	expected result	auto
UX-01	Severity is not colour-only	View the citizen panel and map in a deuteranopia simulator.	Band is still identifiable — the band name is always shown as text beside the value. Map dots currently rely on colour alone.	no
UX-02	Contrast ratios	Run an automated contrast audit on both views.	Body text $\geq 4.5:1$, large text $\geq 3:1$ against the dark ground.	no
UX-03	Keyboard navigation	Traverse the citizen flow with Tab and Enter only.	Ward select, group buttons, language buttons and Get my advisory are all reachable and operable.	no
UX-04	Focus visibility	Tab through both views.	A visible focus ring at every stop.	no
UX-05	Screen-reader labelling	Traverse with a screen reader.	The AQI figure is announced with its band and provenance, not as a bare number.	no
UX-06	Provenance survives magnification	Zoom to 200%.	The ESTIMATED chip and dashed border remain visible; provenance must not be the first thing lost.	no

Responsive and device behaviour

id	validates	procedure	expected result	auto
UX-07	Citizen mode on a phone	Load at 360×640.	Ward selector, value, guidance and Listen all usable without horizontal scrolling. Residents are on phones.	no
UX-08	War-room on a laptop	Load at 1366×768.	Map and rail both usable; action cards not truncated.	no
UX-09	Tap targets	Measure the language and group buttons on mobile.	$\geq 44 \times 44$ px.	no
UX-10	Map absence is survivable	Disable WebGL.	Tabular data still renders. The map is an enhancement, not the only path to the information.	no

States, errors and internationalisation

id	validates	procedure	expected result	auto
UX-11	Loading states	Throttle the network to slow 3G; load both views.	Explicit loading indication; no flash of an empty or zeroed reading that could be misread as data.	no
UX-12	API unreachable	Block the API host; reload.	The static demo fallback engages and the footer says 'static deployment', so no one mistakes a snapshot for live data.	no
UX-13	Partial data	Point at a city with sensors reporting nulls.	Null sensors are excluded, not rendered as 0 or as a band.	no
UX-14	Empty is not error	View a city with no live actions.	Reads as a deliberate answer with a reason, visually distinct from a failure.	no
UX-15	Indic script rendering	View Hindi, Marathi, Kannada advisories on Windows, macOS, Android.	No tofu boxes, no clipped conjuncts, correct line breaking.	no
UX-16	Numerals in translation	Inspect a Marathi advisory.	Whichever numeral system is used, the value equals the English one. ३३३ and 380 are both acceptable; a different number is Sev-1.	no
UX-17	Long ward names	Select the longest ward name in each city.	No overflow, no truncation that hides the landmark suffix.	no
UX-18	Copy accuracy	Read every user-facing claim on both views.	No text asserts a refresh cadence, a measurement, or a legal conclusion the system cannot support.	no

10 · Regression suite — defects that shipped once

Every case here corresponds to a defect that reached production. They are the highest-yield tests in this plan: each represents a failure mode this codebase has actually demonstrated. Most already have an automated assertion; the rest should get one.

id	validates	procedure	expected result	auto
REG-01	band(NaN) returned 'Severe'	Feed a null AQI through the band function and the stations endpoint.	Returns null. No-data sensors were painted blood red.	yes
REG-02	Duplicate station identity	Assert no station_name maps to two station_ids.	163 rows collapsed to 90 real sensors; every live row had lost its ward.	yes
REG-03	Single enforcement pool	Confirm live and episode ranked separately.	December's crisis outranked every current event permanently.	yes
REG-04	Zero-priority actions	Assert every action has priority > 0.	Whole cities of actions tied at 0.00, each citing a statute over Good air.	yes
REG-05	Freshness from a build artefact	Compare the banner with the DB clock.	Banner showed the last successful build — 12 days stale — while the store was 2h old.	yes
REG-06	Timezone skew	Assert max(h) is not in the future.	IST wall-clock compared against a UTC now() skewed every window by 5h30m.	no — add
REG-07	Events tab showed the oldest 80	Check the first card is the newest event.	Tail-and-reverse dropped every recent event.	no — add
REG-08	Metrics blank rows	Assert 48h/72h rows and the inventory table are populated.	The inventory table was excluded from the serverless bundle by .vercelignore.	yes
REG-09	Compare hit the page cap	Assert Delhi's event count is not exactly 300.	Client-side derivation always reported the 300-row cap as the true count.	yes
REG-10	Spend tracker raised after billing	Call budget.record with a stub usage object.	Persists and never raises. It previously discarded paid translations by writing to a read-only path.	yes
REG-11	Validator rejected Devanagari numerals	Validate a correct Hindi translation using ■■■■.	Accepted. Rejection silently served English instead.	yes
REG-12	City mean shown as a ward reading	Assert no single value dominates the ward set.	251 of 290 Delhi wards showed 130 while Anand Vihar measured 448.	yes
REG-13	Estimate described as measured	Assert basis=estimated text omits 'measured'.	The deployed API answered 'measured now' over interpolated values.	yes
REG-14	Ingest workflow installed the wrong requirements	Confirm the pipeline job uses requirements-pipeline.txt.	main.yml failed 19 consecutive runs on missing duckdb/lightgbm/shapely.	no — add

11 · Non-functional — performance, resilience, security

id	validates	procedure	expected result	auto
NFR-01	Ingest inside the function limit	Time POST /ingest.	Completes well inside Vercel's 60 s ceiling; ~15 s is the observed norm.	no
NFR-02	Cold-start latency	Call /health after idle.	Responds within a few seconds; the connection helper reconnects rather than hanging.	no
NFR-03	Connection reuse	Issue repeated requests to a warm instance.	One connection per warm instance, revalidated with select 1; no per-request reconnect cost.	no
NFR-04	IPv6 fallback	Force the direct Supabase host.	Falls back to the IPv4 pooler. Direct resolves IPv6-only and hangs on IPv4-only runners.	no
NFR-05	Evidence pack generation time	Request a pack cold.	Returns inside the function limit; matplotlib rendering is the slowest path.	no
NFR-06	Ingest gap detection	Query hours captured over the last 24.	Gaps are visible and alertable. Currently a person must look — see §13.	no
NFR-07	Upstream outage — data.gov.in	Simulate a failed fetch.	Retries with backoff, then fails cleanly. No partial or corrupt write.	no
NFR-08	Upstream outage — FIRMS	Simulate a fire-fetch failure.	Pipeline continues; the fire layer reports 'archive-only' rather than degrading silently behind a green check.	no
NFR-09	Budget cap enforced	Set llm_spend.usd near the cap; request a translation.	Refused before the call is billed; English template served.	no
SEC-01	No authentication	Request every endpoint with no credentials.	All succeed. Documented open gap — blocks agency pilot. CORS is ¹⁶ .*.	manual
SEC-02	Ingest token strength	Attempt a near-miss token.	401, constant-time comparison, no timing signal.	yes
SEC-03	RLS parity	Check relrowsecurity on all nine tables.	Currently warns. fires and llm_spend are RLS-off with full anon grants — latent, since the anon key is not published, but it makes both world-writable the moment it is. Reported as a warning, not a failure, while the fix is deferred.	yes — warn

SEC-03 detail. The other seven tables have RLS enabled, which renders their identical anon grants inert. On fires and llm_spend the grants are live: SELECT through TRUNCATE. Resetting llm_spend.usd would defeat the spend cap; truncating fires would silently empty a 17,000-row evidence layer. Verified not currently exploitable — the anon key appears in neither the deployed bundle nor the frontend source, and PostgreSQL returns 401 without it. Enabling RLS on both is the fix and breaks nothing, since neither is read through PostgreSQL.

12 · Entry and exit criteria

Entry criteria — testing may begin when

Both automated suites run clean on the candidate build; a fresh parquet backup of all nine tables exists; the deployed API and frontend commit SHAs are recorded and known to each other; and the feed is under two hours old, so freshness-dependent cases are meaningful.

Exit criteria — the build may ship when

#	criterion
1	Zero open Sev-1 or Sev-2 defects. No exceptions, no deferrals.
2	All §4 data invariants pass against the live store, including the seven not yet automated.
3	All §10 regression cases pass. A regression failure means a fixed defect has returned.
4	Every §6 API case has been run against the deployed URL , not only via the suite.
5	The three ward states (§8) verified visually distinct on the deployed site by a second person.
6	An end-to-end audit walk (§5) completed: one action traced to attributions to readings.
7	Open security gaps re-confirmed as documented and accepted in writing by the receiving agency.

Sign-off

role	confirms	signature	date
Backend engineer	§4, §5, §6, §11 executed and recorded		
Frontend engineer	§7, §8, §9 executed on the deployed site		
Reviewer (second pair of eyes)	§8 ward-state distinctness, §5 audit walk		
Receiving agency	§13 limits and open security gaps accepted		

13 · Gaps in testability

Stated plainly. A test plan that does not say what it cannot prove is misleading, and this is the section a receiving agency should read first.

gap	consequence for validation
No staging environment	Every test runs against production data. Destructive and write-path cases must be coordinated and backed up first. A genuine pre-production environment is the single largest improvement available to this plan.
No test fixtures for the agent chain	Most §5 cases are written as fixture-based but no fixture harness exists yet. Until it does, chain correctness is verified against whatever the weather happens to produce — which cannot exercise the enforcement-floor boundary on demand.
Forecast accuracy is unmeasurable at long horizons	A one-week feature window leaves fewer than 50 honest test rows at 48h and 72h, so the code correctly declines to print an accuracy figure. No test can validate what has not been measured. Resolves with accumulated history, not with code.
Ward estimates cannot be ground-truthed	Interpolated ward AQI has no reference value to compare against — that is why the ward has no sensor. Testing can prove the estimator is internally consistent, honestly labelled and correctly masked; it cannot prove the estimate is right. Treat the label, not the number, as the tested property.
No alerting	A failed cron or a stalled feed is currently noticed by a person looking at the banner. Continuous validation between releases does not exist.
No multi-user testing	Without authentication or multi-tenancy there is no permission model to test, and no realistic concurrency scenario. Both become testable only after those are built.

Recommended first three investments

- 1 · Automate the seven missing §4 invariants into `verify_live.py`. They assert referential integrity the database does not enforce, and they are cheap.
- 2 · Build the agent-chain fixture harness so §5 boundary cases (enforcement floor, era isolation, log-odds cap) can run deterministically rather than waiting on weather.
- 3 · Add a deployed-endpoint smoke suite that curls the production API and compares against the TestClient result. That closes the gap in Figure 1 — the one place where a green suite has already told a false story.